AAEC4484/AAEC(STAT)5484: Applied Economic Forecasting

Your Name Here

Homework #5 - Spring 2026

Instructions: Where necessary ensure that your graphs and visuals have proper titles and axis labels. Refer to the output, whenever appropriate, when discussing the results. **Creativity (coupled with relevance) will be rewarded.**

Q1: Classical Multiplicative Decomposition

In this exercise, we will explore the classical decomposition.

1. Return to the `New Private Housing Units Authorized by Building Permits for Virginia (VABPPRIV)` series. **In a single step**, import the data from the `FRED` database using the `quantmod` package. Convert to a `tsibble` object, with the index converted to a `yearmonth` object, and store it as `permit`.

If you are still encountering issues with the `quantmod` package, you can use the direct website link for the `csv` data, as I showed in class.*

2. Produce an `autoplot` of the `permit` series along with a `gg_subseries` plot. Use the `patchwork` package to combine the two plots. **Remember your titles and axis labels.**

- **Briefly** comment on any pattern you observe.
- Xiaoyi suggested that a multiplicative decomposition is appropriate for this series? Do you agree with her assessment? **Remember to explain your reasoning. A simple yes or no will not suffice.**

3. With visuals and our observations in tow, we are ready to perform our decomposition.

Step i. Using an appropriate centered moving average, $m - MA$ model, extract the trend-cycle component, T_t. Recall that we might need to doubly smooth the series to get it to be symmetric.

Present a plot of your trend-cycle element on top of the original data.

Step ii. Using the trend-cycle computed above, compute and plot the **detrended** series

Step iii (a). Using a `TSLM` model, extract the seasonal component, S_t. Also, plot your seasonal component.

Step iii(b). Produce a plot of the actual data (in darkgray) and overlay the `seasonally-adjusted` variable.

Step iv. Lastly, compute and plot the `remainder` component.

Step v. Produce a gridded plot of the data and the three components. Store your plot as `plot1`. Do not present the plot here.

Please order your variables (graphs) as Data » Trend » Seasonal » Remainder.

Hint: I would like to see you

(i) make use of the `pivot_longer()` function to reshape your data. You can select the 4 columns of interest here.
(ii) convert your variable names to `factors` so that `R` respects the "hierarchy" (that I requested) and is not ordered alphabetically in the faceted graphs.
(iii) use the `facet_wrap()` function to stack your plots in a single column (so they resemble the results of the built-in function).
(iv) use the `ggtitle()` function to add the title "My Decomposition" to your plot.

Step v(b). Use the `classical_decomposition()` function, within the `model()` function, to create `R`'s version of the **multiplicative** decomposition of the series. Store this plot as `plot2`. Again, do not present the plot here.

- Add the title "R's Decomposition" to your plot.

Step vi. Present a 1x2 plot of `plot1` and `plot2`.

4. Now, confirm that your decomposition was correct by **plotting the multiplication of the three extracted components (on y-axis)** against the **actual data (on x-axis)**.

- Be sure to add in a 45° line to see how the scatter plot stacks up against this line.
- Produce this plot using the `ggplot` function.
- Get fancy with it. Add a y-axis title that uses the `bquote()` function to display the $\widehat{y}$ symbol.

5. Produce an autoplot() of the `actual` data, permit, and `autolayer()` your computed `trend-cycle` and `seasonally-adjusted` variables.

Ensure that your series are correctly labeled. You might also need to change the colors of each of the series.

Q2: Forecasting Daily Minimum Temperatures with STL Decomposition

In this exercise, we will work with daily minimum temperatures recorded in Melbourne, Australia, spanning January 1981 through December 1990.

1. Load the dataset `daily-min-temperatures.csv` from our Data folder on GitHub and store it as a `tsibble` object called `temp`. The `Date` column should be parsed using `ymd()` and used as the index.
2. Produce an `autoplot()` of the `temp` series. Briefly comment on any patterns you observe. Does the series appear to have a trend? What can you say about the seasonality?
3. Now **attempt** to produce a `gg_season()` plot of `temp`. **Note: You should encounter a `fill_gaps()` error.**

Read the error message carefully. It is telling you that your data contains **implicit gaps** in the time index — meaning at least one date is absent from the dataset (there is no row for it whatsoever). This is different from having a row with a missing (`NA`) value. The error is also helpfully suggesting a potential fix: `fill_gaps()`.

(a) To confirm how many gaps exist and where use the `has_gaps()` and `count_gaps()` functions. Report your findings. How many gaps are there, and on which dates do they fall?

(b) Now fix the issue using the following code, which first inserts the missing dates into the index and then fills the resulting `NA` values using the **last observed value** (carry-forward):

```
#Remember to remove the `eval=FALSE` argument for this code chunk to run.
temp <- temp %>%
  fill_gaps() %>%
  fill(Temp, .direction = "down")
```

Now reproduce the `gg_season()` plot and comment on the seasonal pattern you observe. What does the shape of the seasonal cycle tell you about Melbourne's climate?

4. Your classmate, William, argues that an **additive** decomposition is appropriate for this series. Do you agree with his assessment? Be sure to explain your reasoning clearly — a simple yes or no will not suffice.

(Hint: think carefully about how the magnitude of the seasonal fluctuations behaves relative to the level of the series.)

5. Your colleague is convinced that the X-11 and SEATS would be appropriate decomposition methods for this dataset. You strongly disagree. Briefly explain why STL is the more suitable choice here, drawing on what you know about the limitations of X-11 and SEATS.
6. Now assign the values in `temp` up to and including December 31, 1989 as a training set called `temp.train`. Assign the remaining values to `temp.test`. Produce a plot clearly showing the train/test split.
7. Using `temp.train`, fit an STL decomposition with **fixed seasonal windows** and `robust = TRUE`. Store the result as `fit1` and present the decomposition plot with an appropriate title.

The `STL` function is able to handle complex (multiple layers of) seasonality. You will notice that the `components` function produces two seasonal components: `season_year` and `season_week`. The former captures seasonality that repeats every year, while the latter captures seasonality that repeats every week. The `season_year` component is likely capturing the annual temperature cycle, while the `season_week` component may be capturing any weekly patterns in the data (e.g., differences between weekdays and weekends).

8. Comment on the decomposition. How would you describe the trend-cycle component? Do you observe any seasonality? Do you notice anything peculiar about the remainder component? What could it be signalling here?
9. In a single graph, superimpose the **seasonally adjusted** and **trend-cycle** components onto the training data. Be sure to use distinct (and visible) colors and include an appropriate title.
10. Now fit a second STL model on `temp.train`, this time allowing the seasonal and trend windows to be selected automatically by `R`, with robust = TRUE. Store this as fit2 and present its decomposition plot.

Present the two decompositions (`fit1` and `fit2`) as a side-by-side plot using patchwork. Give each plot an appropriate title and comment on any discernible differences between the two, including any differences in the magnitude of the seasonal component.

11. Using the **fixed seasonality** model in Part 7,

    i. use an random walk with drift model to produce predictions of the **seasonally adjusted** portion of `temp` over the length of the test period. Store your results in a variable called `arima.temp`.

    ii. produce a forecast of each seasonal component (`season_year` and `season_week`) over the test period using the **seasonal naïve** method. Save these as `snaive.seas.year` and `snaive.seas.week` respectively.

    iii. produce a gridded plot (as a 3X1) of the forecasted of both methods over the test period.

12. Combine the forecasts from Part 11 to produce a reseasonalized forecast of the original temperature data. Store this as `pred.temp` and produce a plot displaying the reseasonalized forecast alongside the actual test data (`temp.test`).

Hint: You might find it helpful to (i) convert `drift.temp`, `snaive.seas.year`, and `snaive.seas.week` to tibble objects; (ii) select the `Date` and `.mean` columns from each; (iii) join all three tibbles by the `Date` column; (iv) sum the three `.mean` columns to get the reseasonalized forecast.

13. Briefly discuss your observations from the plot above. How well does this forecast capture the dynamics of the actual test data? Any insight into how the model could be improved?

14. Using the decomposition_model() function and both the fixed and changing seasonality STL models (with robust = TRUE), fit the following six models on temp.train. Store all models in a single model() call as mod.fit, then store the forecasts over the test period as mod.all. The six models are:

- RW with drift (seasonally adjusted) + Seasonal Naïve (both seasonal components), fixed seasonality
- RW with drift (seasonally adjusted) + Seasonal Naïve (both seasonal components), changing seasonality
- RW with drift (seasonally adjusted) + ARIMA (both seasonal components), fixed seasonality
- RW with drift (seasonally adjusted) + ARIMA (both seasonal components), changing seasonality
- ARIMA (seasonally adjusted) + ARIMA (both seasonal components), fixed seasonality
- ARIMA (seasonally adjusted) + ARIMA (both seasonal components), changing seasonality

Note: Please name all six models appropriately. You are not required to plot the forecasts here.

15. Produce a plot of all six model forecasts alongside the **test data**. Turn the prediction intervals off. Which model appears to do the best job visually?

16. Compute forecast accuracy for all six models. Present your results as a `kable` table rounded to 3 decimal places, reporting `RMSE` and `MAPE`. Which model is preferred by each metric? Be sure to explain how you arrived at your conclusion.

17. Using your preferred model from above, produce a residual diagnostic plot using `gg_tsresiduals()` and conduct a Ljung-Box test with a lag of 7. Do the residuals appear to be white noise? Discuss your findings with reference to both the plot and the test result.